



ROBOCUPJUNIOR SOCCER SIMULATION 2026

TEAM DESCRIPTION PAPER

RoboMatrix

Abstract

This paper presents the software architecture and strategy developed by our team for the IranOpen Soccer Simulation 2026. Our primary objective was to overcome the inherent sensor noise and limited field of view characterized by the simulation environment. To achieve this, we developed a centralized-distributed sensor fusion system. Unlike traditional reactive approaches where each agent acts independently, our system designates one robot as a central processing unit. This unit collects observation vectors from all teammates and utilizes linear algebra to solve for the ball's precise position through triangulation. Furthermore, our navigation relies on robust global-to-local coordinate transformations, allowing for precise movement control. The offensive strategy implements a "virtual target" approach, positioning the robot behind the ball to ensure accurate shooting angles, while the defensive logic employs predictive axis-aligned blocking.

1. Introduction

Team

Amirhossein Sheikh Sofla – Team Captain & Lead Programmer

Amirhossein is an experienced developer and robotics specialist with a profound background in Python, C++, and autonomous systems. As the Captain and Lead Programmer of the Salam Dibaji Robotics Department, he has led his team to prestigious international honors, including 1st place finishes at the FIRA World Cup 2024 (Brazil) and FIRA World Cup 2025 (South Korea). His technical expertise spans across image processing for autonomous vehicles, embedded systems, and full-stack development. For this project, he served as the primary architect of the software logic, responsible for implementing the multi-agent communication protocols, sensor fusion algorithms, and the overall team coordination strategy.

2. Project Planning

a. Overall Project Plan

Our goal was to create a team that functions as a single cohesive unit rather than three separate entities.

Phase 1: Foundation. We began by developing a robust mathematical library for handling trigonometry and coordinate system conversions.

Phase 2: Communication. We established a low-latency communication protocol to share robot coordinates and sensor readings.

Phase 3: Sensor Fusion. We implemented a system to merge data from multiple robots to calculate the ball's location, even when it is in a single robot's blind spot.

Phase 4: Strategy. We differentiated the roles into a goalkeeper/coordinator and two strikers, implementing distinct behaviors for each.

b. Integration Plan

We adopted a modular software design. The core mathematical functions and movement controllers were developed as a shared utility library, accessible to all robots. Each robot role inherits from a base class but implements specific behaviors. We used an agile testing approach, verifying the movement logic first, followed by communication, and finally the complex strategy.

3. Software

a. General software architecture

The software is built upon an Object-Oriented Programming (OOP) paradigm. The architecture is divided into three layers:

1. Driver Layer: Handles direct interaction with the simulator's API (motors, GPS, compass).
2. Utility Layer: Contains stateless mathematical functions for geometry, PID control logic, and data serialization.
3. Logic Layer: Defines the specific state machines for the Goalkeeper and Strikers.

We utilize a standard data interchange format to broadcast observing vectors. One robot acts as the "Server," receiving data streams, while the others act as "Clients," broadcasting their local perception.

b. Navigation

Our navigation system is designed to be independent of the robot's current orientation. It consists of two main components:

1. Global-to-Local Transformation: Since the game strategy relies on absolute field coordinates (Global Frame), but the motors require relative commands (Local Frame), we implemented a rotation matrix transformation. By using the robot's current heading from the compass, we project the target's global coordinates into the robot's local x and y axes.
2. Proportional Control: Once the target is converted to the local frame, we use a proportional controller. The forward velocity is determined by the distance to the target, while the rotational velocity is determined by the angular error. This ensures smooth deceleration as the robot approaches its destination and prevents oscillation.

c. Object Detection & Sensor Fusion

The most innovative aspect of our code is the Multi-Agent Triangulation System. In the simulation, individual ball sensors are subject to noise and blind spots. To solve this, we treat the ball's position not as a sensor reading, but as the solution to a system of linear equations.

1. Data Collection: Each robot calculates a vector pointing toward the ball based on its local sensor data and broadcasts this vector along with its own global position.
2. Mathematical Modeling: Each observation is treated as a line equation in 2D space ($y = mx + c$).
3. Linear Algebra Solution: The central robot aggregates these lines. It constructs a matrix representing the system of equations and solves for the intersection point ($Ax = B$). This approach

allows us to pinpoint the ball's location with high precision, even if the robot closest to the ball has a noisy sensor, by relying on the geometry provided by distant teammates.

d. Strategy & Game State Estimation

Our strategy focuses on dynamic positioning rather than static mapping.

1. The "Virtual Target" Offense: A common issue in robot soccer is robots approaching the ball directly and accidentally knocking it sideways. We implemented a "Virtual Target" algorithm. Instead of navigating to the ball, the striker navigates to a calculated point directly *behind* the ball relative to the opponent's goal. This creates a vector of approach that naturally aligns the robot for a shot towards the goal.
2. Predictive Goalkeeping: The goalkeeper does not merely follow the ball. It constrains its movement to the goal line. It calculates the slope of the ball's movement vector to predict the exact intersection point with the goal line. This allows the goalkeeper to move to the blocking position before the ball arrives.
3. Defensive Fallback: If the sensor fusion system fails or the ball is not visible to any robot for a set duration, the strikers automatically retreat to pre-defined defensive coordinates to protect our side of the field.

4. Performance evaluation

We evaluated our system through two primary tests:

1. **Localization Accuracy Test:** We placed the ball at fixed coordinates and compared the position estimated by a single robot versus our multi-agent triangulation system. The results showed that the triangulation method reduced position error significantly, especially at long distances.
2. **Scoring Efficiency:** We ran matches using a direct-approach strategy versus our "Virtual Target" strategy. The Virtual Target approach resulted in a higher percentage of shots on goal because the robots aligned themselves before striking, rather than pushing the ball into corners.

5. Conclusion

In conclusion, our team successfully implemented a cooperative multi-agent system. By shifting the complexity from individual behavior to shared mathematical processing, we overcame the limitations of individual sensors. The combination of linear algebra for localization and vector-based navigation for strategy provided a robust foundation for gameplay. Future improvements will focus on adding a Kalman Filter to smooth the predicted ball trajectory further.

References

- Webots User Guide - Cyberbotics.
- NumPy Documentation (Linear Algebra routines).
- RoboCupJunior Soccer Simulation Rules 2021.